LoadScope: Phase 0 Architecture

Complete Line-by-Line Code & Engineering Guide

An exhaustive walkthrough of every file, line of code, configuration, and tool in Phase 0.

What is Phase 0?

Phase 0 is the foundation of any professional software project. In this phase, we do not write load-testing logic yet. Instead, we establish the development environment, module systems, compilation pipelines, testing framework, continuous integration, and a controllable mock HTTP server to simulate real-world conditions.

Ø=Ü¡ Phase 0 Definition of Done (Checkpoint)
Phase 0 is successful when: (1) Package name is verified on npm, (2) TypeScript builds to dist/ with zero errors, (3) Mock server tests pass in Vitest, and (4) CI runs green in GitHub Actions.

Files Created in Phase 0

Every file created in Phase 0 serves a distinct, vital purpose in the project lifecycle:

- **package.json:** Defines dependencies, binary paths, npm distribution files, and build scripts.
- **tsconfig.json:** Configures the TypeScript compiler for native Node.js ES Modules (ESM).
- **tsup.config.ts:** Fast esbuild bundler configuration that compiles TypeScript to dist/ with type declarations.
- **vitest.config.ts:** High-speed native ESM test runner configuration.
- **src/types.ts:** Defines the core data contracts (RunOptions, LatencyPoint, RunSummary).
- **src/index.ts:** Programmatic library entrypoint allowing other developers to use LoadScope in Node.js scripts.
- **src/cli.ts:** Executable command-line interface entrypoint using Commander and Picocolors.
- **test-server/index.ts:** Controllable native HTTP server with delay, error, and stall endpoints.
- **test/test-server.test.ts:** Vitest integration test verifying that the mock server responds as expected.
- **.github/workflows/ci.yml:** Automated CI pipeline testing on Node 20 and Node 22 on every commit.

1. package.json — Line-by-Line Breakdown

The `package.json` file is the central manifest of our project. It dictates how Node.js runs our code, how npm installs it, and what dependencies are bundled.

```
{
  "name": "loadscope",
  "version": "0.1.0",
  "description": "Load tester with breaking-point finder, CI regression gate, and PDF reports",
  "type": "module",
  "bin": {
    "loadscope": "dist/cli.js"
  },
  "main": "dist/index.js",
  "types": "dist/index.d.ts",
  "files": [
    "dist",
    "README.md",
    "LICENSE"
  ],
  "license": "MIT",
  "engines": {
    "node": ">=20"
  },
  "scripts": {
    "build": "tsup",
    "test": "vitest run",
    "test:watch": "vitest",
    "test-server": "node --loader tsx test-server/index.ts",
    "prepublishOnly": "npm run build && npm test"
  }
}
```

- **"name": "loadscope":** The exact package identifier on npm. We verified with `npm view loadscope` (404) that this name is free to take.
- **"version": "0.1.0":** Semantic Versioning (SemVer): Major.Minor.Patch. 0.1.0 indicates initial pre-1.0 development.
- **"type": "module":** Forces Node.js to treat all .js files as ECMAScript Modules (ESM). This enables modern `import` and `export` statements.
- **"bin": { "loadscope": "dist/cli.js" }:** Registers the global command. When an end user runs `npx loadscope`, npm executes `dist/cli.js` directly.
- **"main": "dist/index.js":** The programmatic entrypoint. When another project does `import { run } from 'loadscope'`, Node resolves this file.
- **"types": "dist/index.d.ts":** Points TypeScript to the auto-generated type definitions, giving full IDE autocomplete to users.
- **"files": [...]:** Whitelist of files uploaded to npm registry. Keeps the published package lightweight by excluding src, tests, and configs.
- **"engines": { "node": ">=20" }:** Ensures users run Node.js 20 LTS or newer, which is required for undici connection pooling and worker_threads stability.
- **"scripts.build":** Executes tsup to bundle TypeScript files into dist/.
- **"scripts.test":** Executes Vitest in single-run mode for CI pipelines.
- **"scripts.prepublishOnly":** Safety hook: Automatically builds and runs all unit tests before npm allows publishing a new release.

2. tsconfig.json & tsup.config.ts

TypeScript Configuration (tsconfig.json)

Configures TypeScript for strict type checking and native Node ESM output.

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "lib": ["ES2022"],
    "outDir": "dist",
    "rootDir": "src",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true,
    "declaration": true,
    "declarationMap": true,
    "sourceMap": true,
    "resolveJsonModule": true
  },
  "include": ["src/**/*"],
  "exclude": ["node_modules", "dist", "test", "test-server"]
}
```

- **"target": "ES2022":** Emits modern ECMAScript syntax (class fields, top-level await, Array.prototype.at) supported natively by Node 20+.
- **"module": "NodeNext":** Matches Node's official ECMAScript Module standard. Requires explicit file extensions in imports (e.g. `./types.js`).
- **"moduleResolution": "NodeNext":** Uses Node's modern resolution algorithm that respects the `exports` and `imports` fields in package.json.
- **"strict": true:** Enables all strict type checks: noImplicitAny, strictNullChecks, and strictFunctionTypes, catching bugs at compile time.
- **"declaration": true:** Instructs the compiler to output `.d.ts` declaration files so library consumers get full typing support.
- **"skipLibCheck": true:** Skips type checks on declaration files in node_modules, significantly reducing compilation time.

Bundler Configuration (tsup.config.ts)

```
import { defineConfig } from "tsup";

export default defineConfig({
  entry: ["src/cli.ts", "src/index.ts"], // Bundles both entrypoints
  format: ["esm"],                       // Emits modern ES Module syntax
  dts: true,                             // Emits dist/cli.d.ts and dist/index.d.ts
  clean: true,                           // Deletes dist/ directory before rebuilding
  sourcemap: true,                       // Generates .map files for debugging
  target: "node20",                      // Optimizes output for Node.js 20+
  shims: true,                           // Injects ESM polyfills (import.meta.url)
});
```

3. Types & Entrypoints (src/types.ts & src/index.ts)

Core Type Contracts (src/types.ts)

These interfaces define the data shape throughout the load tester, from CLI arguments to final results.

```
export interface RunOptions {
  url: string;                      // Target web server address
  duration?: number;                // Duration in seconds (e.g., 30)
  concurrency?: number;             // Concurrent workers (closed model)
  rate?: number;                    // Requests per second (open model)
  method?: string;                  // GET, POST, PUT, DELETE, etc.
  headers?: Record<string, string>;// Custom HTTP headers
  body?: string;                    // Request body for POST/PUT
  pdf?: string;                     // Filepath to generate PDF report
  aiReport?: string;                // Filepath for PDF with AI analysis
  aiDryRun?: boolean;               // Print prompt without sending to LLM
}

export interface LatencyPoint {
  t: number;                        // Elapsed second offset (0, 1, 2...)
  p95: number;                      // 95th percentile latency at that second
}

export interface RunSummary {
  target: string;                   // The URL tested
  rps: number;                      // Measured requests per second
  totalRequests: number;            // Total dispatched requests
  durationSec: number;              // Actual test duration
  p50: number;                      // Median latency (50th percentile) in ms
  p95: number;                      // 95th percentile latency in ms
  p99: number;                      // 99th percentile latency in ms
  errorRate: number;                // Ratio of failed requests (0.00 to 1.00)
  statusCodes: Record<number, number>; // Map of HTTP codes { 200: 4500, 500: 12 }
  series: LatencyPoint[];           // Time series for charts (max 60 points)
  selfCheckWarnings?: string[];     // Warnings if LoadScope CPU/event loop lagged
}
```

Programmatic API (src/index.ts)

```
import type { RunOptions, RunSummary } from "./types.js";
export * from "./types.js";

export async function run(options: RunOptions): Promise<RunSummary> {
  // Phase 0 placeholder: returns an initial summary stub
  return {
    target: options.url,
    rps: 0,
    totalRequests: 0,
    durationSec: options.duration ?? 10,
    p50: 0,
    p95: 0,
    p99: 0,
    errorRate: 0,
    statusCodes: {},
    series: [],
  };
}
```

4. CLI Entrypoint (src/cli.ts)

The CLI entrypoint handles flag parsing, user validation, and visual formatting in the terminal. When run via `npx loadscope`, Node executes this file.

```
#!/usr/bin/env node
import { Command } from "commander";
import picocolors from "picocolors";
import { run } from "./index.js";

const program = new Command();

program
  .name("loadscope")
  .description("A high-performance load-testing CLI with breaking-point finder and CI regression gate")
  .version("0.1.0")
  .option("-u, --url <url>", "Target URL to test")
  .option("-c, --concurrency <number>", "Number of concurrent connections", "10")
  .option("-d, --duration <duration>", "Test duration (e.g. 10s, 1m)", "10s")
  .option("-r, --rate <number>", "Requests per second (open arrival model)")
  .option("--pdf <path>", "Generate a PDF report at the specified path")
  .option("--ai-report <path>", "Generate a PDF report including AI analysis (requires LLM_API_KEY)")
  .option("--ai-dry-run", "Preview prompt that would be sent to LLM without calling API")
  .action(async (options) => {
    if (!options.url) {
      console.error(picocolors.red("Error: --url is required"));
      process.exit(1);
    }

    console.log(picocolors.cyan(`\nStarting LoadScope test against
${picocolors.bold(options.url)}...`));
    console.log(picocolors.dim(`Concurrency: ${options.concurrency} | Duration: ${options.duration}`));

    const result = await run({
      url: options.url,
      concurrency: parseInt(options.concurrency, 10),
      pdf: options.pdf,
      aiReport: options.aiReport,
      aiDryRun: options.aiDryRun,
    });

    console.log(picocolors.green("Test completed successfully."));
  });

program.parse(process.argv);
```

- **"#!/usr/bin/env node":** The Shebang. Instructs Unix/Linux/macOS shells to execute this file using the Node.js runtime.
- **"import { Command } from 'commander'":** Loads Commander, the battle-tested library for parsing CLI flags, arguments, and displaying `--help`.
- **"import picocolors from 'picocolors'":** Ultra-lightweight terminal color library (much faster and smaller than Chalk).
- **"program.option(...)":** Defines supported CLI options with shorthand aliases (`-u`, `-c`, `-d`), descriptions, and default values.
- **"if (!options.url)":** Validation guard: Halts execution with an informative red error message if the user omitted the target URL.

5. Controllable Test Server (test-server/index.ts)

To write repeatable unit and integration tests, we need a controllable target server built with zero external dependencies using Node's native `http` module.

```
import http from "node:http";

export interface TestServerOptions {
  port?: number;
  host?: string;
}

export function createTestServer() {
  const server = http.createServer((req, res) => {
    const url = new URL(req.url ?? "/", `http://${req.headers.host ?? "localhost"}`);
    const pathname = url.pathname;

    // Fast 200 OK: Baseline endpoint
    if (pathname === "/" || pathname === "/health") {
      res.writeHead(200, { "Content-Type": "application/json" });
      res.end(JSON.stringify({ status: "ok", timestamp: Date.now() }));
      return;
    }

    // Controllable fixed latency: /delay/:ms or /delay?ms=100
    if (pathname.startsWith("/delay")) {
      const parts = pathname.split("/").filter(Boolean);
      let ms = 100;
      if (parts.length >= 2 && !isNaN(Number(parts[1]))) {
        ms = Number(parts[1]);
      } else if (url.searchParams.has("ms")) {
        ms = Number(url.searchParams.get("ms"));
      }

      setTimeout(() => {
        if (!res.writableEnded) {
          res.writeHead(200, { "Content-Type": "application/json" });
          res.end(JSON.stringify({ delayed: ms, timestamp: Date.now() }));
        }
      }, ms);
      return;
    }

    // Controllable error response: /error or /error?status=503
    if (pathname.startsWith("/error")) {
      const statusCode = Number(url.searchParams.get("status") ?? 500);
      res.writeHead(statusCode, { "Content-Type": "application/json" });
      res.end(JSON.stringify({ error: "Internal Error Simulation", status: statusCode }));
      return;
    }

    // Stall endpoint: leaves socket open without responding (tests timeouts)
    if (pathname === "/stall") {
      const stallMs = Number(url.searchParams.get("ms") ?? 30000);
      const timer = setTimeout(() => {
        if (!res.writableEnded) {
          res.writeHead(200, { "Content-Type": "application/json" });
          res.end(JSON.stringify({ stalled: stallMs }));
        }
      }, stallMs);

      req.on("close", () => clearTimeout(timer));
      return;
    }

    // Fallback 404
    res.writeHead(404, { "Content-Type": "application/json" });
    res.end(JSON.stringify({ error: "Not Found", path: pathname }));
  });

  return server;
}
```

6. Test Suite & CI Pipelines

Vitest Integration Test (test/test-server.test.ts)

Verifies that the mock server works as intended before any load test features are developed.

```
import { describe, it, expect, beforeAll, afterAll } from "vitest";
import http from "node:http";
import { startTestServer } from "../test-server/index.js";

describe("LoadScope Controllable Test Server", () => {
  let serverInstance: http.Server;
  let baseUrl: string;

  beforeAll(async () => {
    // Port 0 instructs the OS to assign a random available port (prevents port conflicts)
    const result = await startTestServer({ port: 0 });
    serverInstance = result.server;
    baseUrl = result.url;
  });

  afterAll(async () => {
    await new Promise<void>((resolve, reject) => {
      serverInstance.close((err) => (err ? reject(err) : resolve()));
    });
  });

  it("responds with 200 OK for root endpoint", async () => {
    const res = await fetch(`${baseUrl}/`);
    expect(res.status).toBe(200);
    const body = await res.json();
    expect(body.status).toBe("ok");
  });

  it("handles controllable latency on /delay/:ms", async () => {
    const start = Date.now();
    const delay = 50;
    const res = await fetch(`${baseUrl}/delay/${delay}`);
    const elapsed = Date.now() - start;

    expect(res.status).toBe(200);
    expect(elapsed).toBeGreaterThanOrEqual(delay - 10);
    const body = await res.json();
    expect(body.delayed).toBe(delay);
  });

  it("returns error code for /error endpoint", async () => {
    const res = await fetch(`${baseUrl}/error?status=503`);
    expect(res.status).toBe(503);
  });
});
```

GitHub Actions CI (.github/workflows/ci.yml)

```yaml
name: ci
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        node-version: [20.x, 22.x]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: ${{ matrix.node-version }}
      - run: npm ci
      - run: npm run build
      - run: npm test
```